TYPES · WHERE SILENT BUGS HIDE

The Type Error That Makes Your Analysis *Wrong* Without Crashing

Audit your own scripts for silent type bugs, then rebuild Tutorial 1's average as two reusable, type-safe functions.

NAME

DATE

THRESHOLD USED

1 Your Last Analysis — Type Audit

→Open a recent script. List **3 variables** — name each, then run `type()` on it.
→For each: what you *assumed* vs what `type()` **actually returned**.
→Did a column arrive as `str` when you expected numbers? Caught by error, wrong output, or **never**?
→Write one line where `"28" + "35"`-style concatenation could've silently hit your work.

```
var_1 = ___   assumed: ___   type(): ___
```

ASSUMED VS ACTUAL

2 Lists vs Dicts — What Are You Actually Storing?

→Employee records: `name`, `salary`, `department`. **List of dicts** or flat salary list? One sentence why.
→From *memory*, write a comprehension filtering salaries above your own threshold. Verify after.
→Name **one scenario** where list-of-dicts becomes wrong tool — what do you reach for instead?

```
high_paid = [e for e in employees if e["salary"] > ____]
```

CHOSEN THRESHOLD

3 Function Refactor — Your Duplicated Code

→Find **one copy-pasted block** across your notebooks/scripts. Describe or paste it here.
→Rewrite as a **named function**, one parameter minimum, plus a sensible `default=`.
→What breaks if *wrong type* gets passed in? How would you catch it — `type()` check, `try/except`, assert?

```
def my_function(data, threshold=____):
    ...
```

DUPLICATED BLOCK FOUND

4 The Four Primitives — Intention Check

→One real example from **your own data work** for each: `int` / `float` / `str` / `bool`.
→Where have you mixed `int` and `float` without thinking? What was the *result*?
→Fix this: `age_column = ['28', '35', '42']` — you need to **sum it**. Write the conversion.

```
age_column = ['28', '35', '42']
total = sum(int(a) for a in age_column)
```

INT / FLOAT / STR / BOOL

5 Build Your Personal Type-Safety Checklist

→List **3 columns** you work with regularly. Expected type vs what Python *often loads* them as.
→Write a **3-step pre-analysis check** to run before touching any new CSV.
→Rewrite Tutorial 1's average as `calculate_average()`. Add `is_outlier(value, data, threshold=2.0)` returning `True/False`. Paste it.

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)
```

COLUMN → EXPECTED TYPE